Experiment No. 6

To automate application deployment using GitHub and Jenkins

1. Aim

To automate application deployment using GitHub and Jenkins.

2. Objectives

- Understand automated application deployment using CI/CD.
- Integrate a Flask application with GitHub.
- Automate build, test and deployment using GitHub Actions.
- Automate build, test and deployment using Jenkins.
- Deploy the Flask application using Docker.

3. Theory

Continuous Integration and Continuous Deployment (CI/CD) automate the process of building, testing and deploying an application. In this experiment, a simple Flask application is packaged using Docker. Part 1 uses GitHub Actions to automatically execute the workflow when code is pushed to the main branch. Part 2 uses Jenkins to check out the GitHub repository, test the application, build a Docker image and deploy it in a Docker container.

4. Requirements

- Computer/Laptop
- Windows/Linux
- Python 3.11
- Git
- Docker Desktop
- GitHub account and repository
- Jenkins server
- Web browser

5. Project Structure

```
student-flask-app/
├── app.py
├── requirements.txt
```

```
├── Dockerfile
├── Jenkinsfile
└── .github/
    └── workflows/
        └── deploy.yml
```

PART 1 – AUTOMATED DEPLOYMENT USING GITHUB

6. Create Flask Application

Create app.py:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello! Application deployed automatically."

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

7. Create requirements.txt

```
Flask
```

8. Create Dockerfile

```
FROM python:3.11
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

9. Test Application Using Docker

Build the image:

```
docker build -t student-flask-app .
```

Run the container:

```
docker run -d -p 5000:5000 --name student-app student-flask-app
```

Check the container:

```
docker ps
```

Open in browser:

```
http://localhost:5000
```

Expected output:

```
Hello! Application deployed automatically.
```

Stop and remove the test container:

```
docker stop student-app
docker rm student-app
```

10. Create GitHub Repository

Create a repository named student-flask-app and push the project:

```
git init
git add .
git commit -m "Add Flask Docker application"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/student-flask-app.git
git push -u origin main
```

11. Create GitHub Actions Workflow

Create .github/workflows/deploy.yml:

```
name: Build and Deploy Flask Application

on:
  push:
    branches:
      - main

jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install Dependencies
        run: pip install -r requirements.txt

      - name: Test Application
        run: python -m py_compile app.py

      - name: Build Docker Image
        run: docker build -t student-flask-app .

      - name: Run Docker Container
        run: docker run -d -p 5000:5000 --name student-app student-flask-app
```

```
      - name: Verify Container
        run: docker ps

      - name: Deployment Successful
        run: echo "Application deployed successfully"
```

12. Push Workflow to GitHub

```
git add .
git commit -m "Add GitHub Actions deployment"
git push origin main
```

13. Verify GitHub Actions

Open GitHub → Repository → Actions. Open the workflow and verify:

```
Checkout Code            ✓
Set up Python            ✓
Install Dependencies     ✓
Test Application         ✓
Build Docker Image       ✓
Run Docker Container     ✓
Verify Container         ✓
Deployment Successful    ✓
```

14. Automatic Deployment Test

Modify the response in app.py, then commit and push the change:

```
git add .
git commit -m "Update application message"
git push origin main
```

A push to main automatically starts the GitHub Actions workflow.

PART 2 – AUTOMATED DEPLOYMENT USING JENKINS

15. Start Jenkins

Start Jenkins and open:

```
http://localhost:8080
```

Log in to Jenkins. Jenkins must have access to Git, Python and Docker. A local Jenkins installation must also have permission to execute Docker commands.

16. Create Jenkins Pipeline Job

Jenkins Dashboard → New Item → enter student-flask-deployment → select Pipeline → OK.

17. Configure Pipeline from GitHub

In Pipeline configuration select Pipeline script from SCM, choose Git, enter the repository URL, set the branch to */main and Script Path to Jenkinsfile.

```
Repository:
https://github.com/YOUR-USERNAME/student-flask-app.git

Branch:
*/main

Script Path:
Jenkinsfile
```

18. Create Jenkinsfile

Create Jenkinsfile in the root of the repository:

```
pipeline {
    agent any

    stages {

        stage('Checkout Code') {
            steps {
                checkout scm
            }
        }

        stage('Install Dependencies') {
            steps {
                bat 'python -m pip install -r requirements.txt'
            }
        }

        stage('Test Application') {
            steps {
                bat 'python -m py_compile app.py'
            }
        }

        stage('Build Docker Image') {
            steps {
                bat 'docker build -t student-flask-app .'
            }
        }

        stage('Stop Old Container') {
```

```
            steps {
                bat 'docker stop student-app || exit /b 0'
                bat 'docker rm student-app || exit /b 0'
            }
        }

        stage('Deploy Application') {
            steps {
                bat 'docker run -d -p 5000:5000 --name student-app student-flask-app'
            }
        }

        stage('Verify Deployment') {
            steps {
                bat 'docker ps'
            }
        }
    }

    post {
        success {
            echo 'Application deployed successfully using Jenkins.'
        }

        failure {
            echo 'Application deployment failed.'
        }
    }
}
```

19. Push Jenkinsfile to GitHub

```
git add Jenkinsfile
git commit -m "Add Jenkins deployment pipeline"
git push origin main
```

20. Run Jenkins Pipeline

Open Jenkins → student-flask-deployment → Build Now. Open the build and select Console Output to observe the execution.

```
Checkout Code
      ↓
Install Dependencies
      ↓
Test Application
      ↓
Build Docker Image
      ↓
Stop Old Container
      ↓
Deploy Application
      ↓
Verify Deployment
```

21. Verify Jenkins Deployment

Check the running container:

```
docker ps
```

Open:

```
http://localhost:5000
```

Expected output:

```
Hello! Application deployed automatically.
```

22. Automate Jenkins on Git Push

Jenkins can be configured with a GitHub webhook or SCM polling so that a push to the repository triggers the pipeline automatically.

```
git add .
git commit -m "Update application"
git push origin main
```

23. Observation

The Flask application was integrated with GitHub. GitHub Actions automatically checked out, tested, built and ran the Docker application. Jenkins performed the same CI/CD stages using a Jenkinsfile.

24. Result

The application was successfully deployed automatically using GitHub Actions and Jenkins with Docker.

25. Conclusion

This experiment demonstrated automated application deployment using GitHub Actions and Jenkins. GitHub Actions provides GitHub-integrated workflow automation, while Jenkins provides flexible pipeline-based automation. Docker was used to package and run the application consistently.

26. Viva Questions

What is CI/CD?

CI/CD automates code integration, testing, build and deployment.

What is GitHub Actions?

A GitHub-integrated workflow automation service.

What is Jenkins?

An automation server commonly used for CI/CD pipelines.

What is a Jenkinsfile?

A file that defines a Jenkins pipeline as code.

Why is Docker used?

It packages the application and dependencies into a container.

What triggers the GitHub Actions workflow?

A push to the main branch in this experiment.

What is the purpose of the Docker build stage?

It creates the Docker image for the application.

How can Jenkins be triggered automatically?

Using a GitHub webhook or SCM polling.